

Hushh Protocol

Send silently.

A zero-knowledge payment rail for USDC on Arc. Hushh breaks the on-chain link between who paid and who was paid — using a shielded note pool, Poseidon commitments, and Groth16 proofs of Merkle membership — while keeping the experience as simple as sending a message to an **@username**.

NETWORK	CHAIN ID	ASSET	PROOF SYSTEM	STATUS
Arc Testnet	5042002	USDC	Groth16 / BN254	Live on testnet

01 Abstract

Every transfer on a public blockchain permanently publishes an edge: `sender → recipient`. For payroll, donations, remittances and vendor payments, that edge is not incidental metadata — it *is* the sensitive information. Hushh Protocol removes it.

Hushh is a **shielded note pool** deployed on Arc, an EVM chain with USDC as its native settlement currency. A sender deposits a fixed-denomination note under a Poseidon commitment. An indexer inserts that commitment into a Merkle tree and posts the root on-chain. Later, the recipient withdraws by submitting a Groth16 proof that their note belongs to *some* known root, revealing only a nullifier that prevents double-spend. No on-chain data connects the deposit transaction to the withdraw transaction.

What distinguishes Hushh from prior shielded pools is not the cryptography — that lineage is well understood — but the layer above it. Existing privacy tooling is cryptographically strong and operationally hostile: raw note strings, manual backups, no identity, no delivery. Hushh adds `@username` resolution, an end-to-end encrypted note inbox, and a browser-native proving SDK, so that a private payment requires no more of the user than a public one.

Contents

01 Abstract	2
02 The transparency problem	3
03 Why Arc	3
04 Protocol overview	4
05 System architecture	5
06 Deposit & withdraw flow	6
07 Cryptographic construction	7
08 Contracts & state machine	8
09 Trust model & threat analysis	9
10 Honest privacy limits	10
11 Deployment	10
12 Roadmap	11
13 Design principles	11

Reading this document. Sections 02–04 are non-technical and describe what Hushh does and why. Sections 05–09 specify the system: architecture, flows, circuit, and trust boundaries. Section 10 states — plainly and without hedging — what Hushh does *not* protect.

02 The transparency problem

Public chains are transparent by construction. That property is a feature for settlement assurance and a liability for human relationships. Once a payment is made, the association between two parties is permanent, globally readable, and trivially clusterable.

USE CASE	WHAT THE CHAIN LEAKS
Payroll	Each contributor's exact compensation, payment cadence, and their affiliation with the paying entity — visible to competitors, counterparties, and the contributor's peers.
Donations	A public, permanent list of supporters for political, legal, or humanitarian causes — in jurisdictions where that list may be used against them.
Remittances	Recipients in a single corridor become an enumerable, targetable cohort with known balances and known arrival times.
Commerce	Suppliers, volumes, margins, and payment terms — disclosed to any counterparty before a negotiation begins.

Why the usual mitigations fail

- **Fresh addresses.** Defeated by funding-trail analysis: the new address must be funded from somewhere, and that somewhere is the link.
- **Custodial intermediaries.** Reintroduce exactly the counterparty trust that on-chain settlement was adopted to remove, and concentrate a deanonymization target.
- **Existing shielded pools.** Cryptographically adequate but unusable at scale by non-specialists — no identity layer, no delivery mechanism, no recovery path.

The gap Hushh targets: unlinkability that a non-cryptographer can operate correctly on the first attempt, denominated in the stablecoin people actually hold.

03 Why Arc

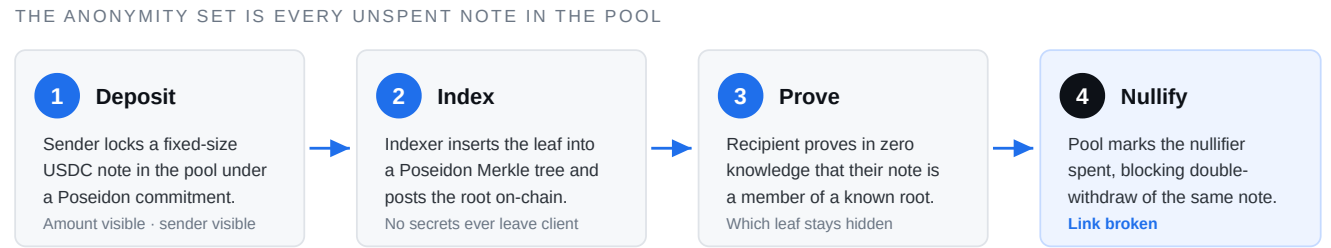
Hushh is deployed on Arc because the chain's design assumptions align unusually well with a stablecoin privacy rail.

PROPERTY	CONSEQUENCE FOR HUSHH
USDC as the native gas and settlement asset	Users hold and reason in one unit. No secondary gas token to acquire before a private payment, and no volatile denominator complicating fixed-note sizing.
Full EVM equivalence	Solidity contracts, Hardhat tooling, and the standard Groth16 verifier pattern port directly. The BN254 precompiles required for on-chain pairing checks are available at predictable cost.
Predictable, low fees	Groth16 verification is a fixed, non-trivial gas cost. Fee predictability is what makes per-payment shielding economically sensible rather than a luxury reserved for large transfers.
Fast, deterministic finality	The indexer can post Merkle roots promptly and without reorg-handling complexity, keeping the deposit-to-withdrawable latency short.
Payments-oriented ecosystem	Anonymity is a function of pool size. A chain whose dominant activity is stablecoin payments is the environment where a USDC-denominated anonymity set can realistically grow.

The last point is the load-bearing one. A shielded pool's privacy guarantee scales with the number of unspent notes it holds; deploying on a general-purpose chain where payments are a minority of activity produces a thin set. Arc's concentration on stablecoin settlement is what makes the anonymity set plausible.

04 Protocol overview

Hushh is a shielded note pool with an identity and delivery layer built on top of it. The protocol proper is four steps; everything else exists to make those four steps usable.



From the chain's perspective, a withdrawal could have originated from any unspent note — the larger the pool, the weaker any correlation.

Figure 1 — The four-step protocol lifecycle.

The usability layer

Steps 1–4 are sufficient for privacy and insufficient for adoption. Three additional components carry the user experience, none of which are trusted with funds or secrets.

LAYER	FUNCTION	WHAT IT CAN SEE
Username registry	Resolves <code>@handle</code> to a wallet address and a Hushh encryption public key, so senders address humans rather than hex.	Public directory data only
Encrypted inbox	Note payloads are sealed to the recipient's Hushh public key and stored server-side as opaque ciphertext, then delivered on login.	Ciphertext and routing metadata
Proof SDK	Generates secrets, commitments, and nullifiers; assembles Merkle witnesses; produces Groth16 proofs entirely in the browser.	Runs client-side only

Invariant. The backend stores ciphertext and routing metadata. It never holds a secret, never generates a proof, and cannot spend a note. A total compromise of the Hushh backend leaks correspondence metadata — it does not leak funds and does not break on-chain unlinkability.

05 System architecture

Six layers, with a hard boundary between what runs on the client and what runs on a server. Every secret lives above the boundary; every server below it is a liveness dependency, never a safety one.

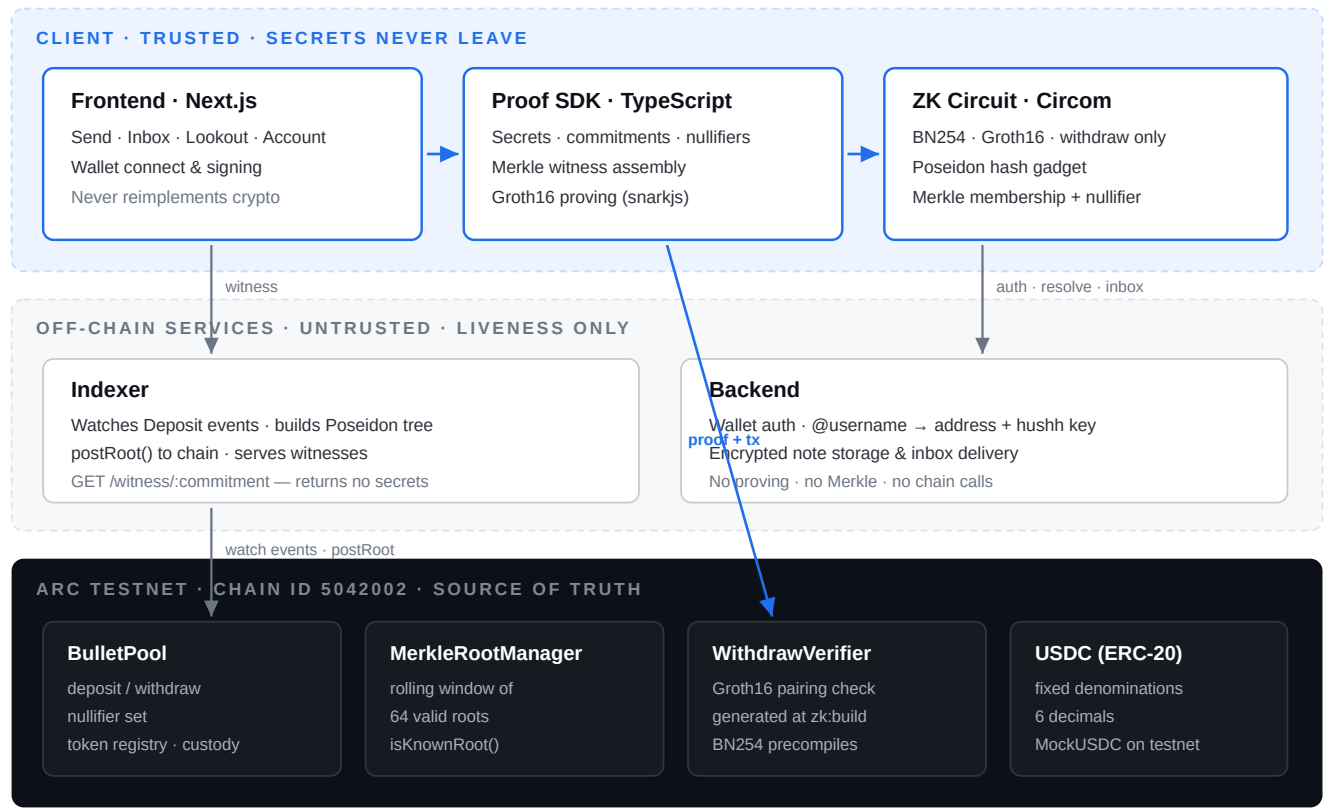


Figure 2 — Layered architecture and trust zones. Blue arrows carry proofs; grey arrows carry public data only.

COMPONENT	RESPONSIBILITY	FAILURE MODE
Smart contracts	Hold funds, verify Groth16 proofs, track nullifiers, accept and window roots	Safety-critical
Indexer	Watch deposits, maintain the Poseidon Merkle tree, post roots, serve witnesses	Liveness only
SDK	Secrets, commitments, nullifiers, Merkle proofs, Groth16 proving	Client-side, safety-critical
Backend	Wallet auth, username resolution, encrypted inbox	Liveness & metadata
Frontend	Deposit and withdraw UX over SDK, backend, and contracts	Liveness only

06 Deposit & withdraw flow

The two paths never touch. Nothing produced during deposit is referenced during withdraw except a Merkle root shared by hundreds of other notes — which is precisely why the link cannot be reconstructed.

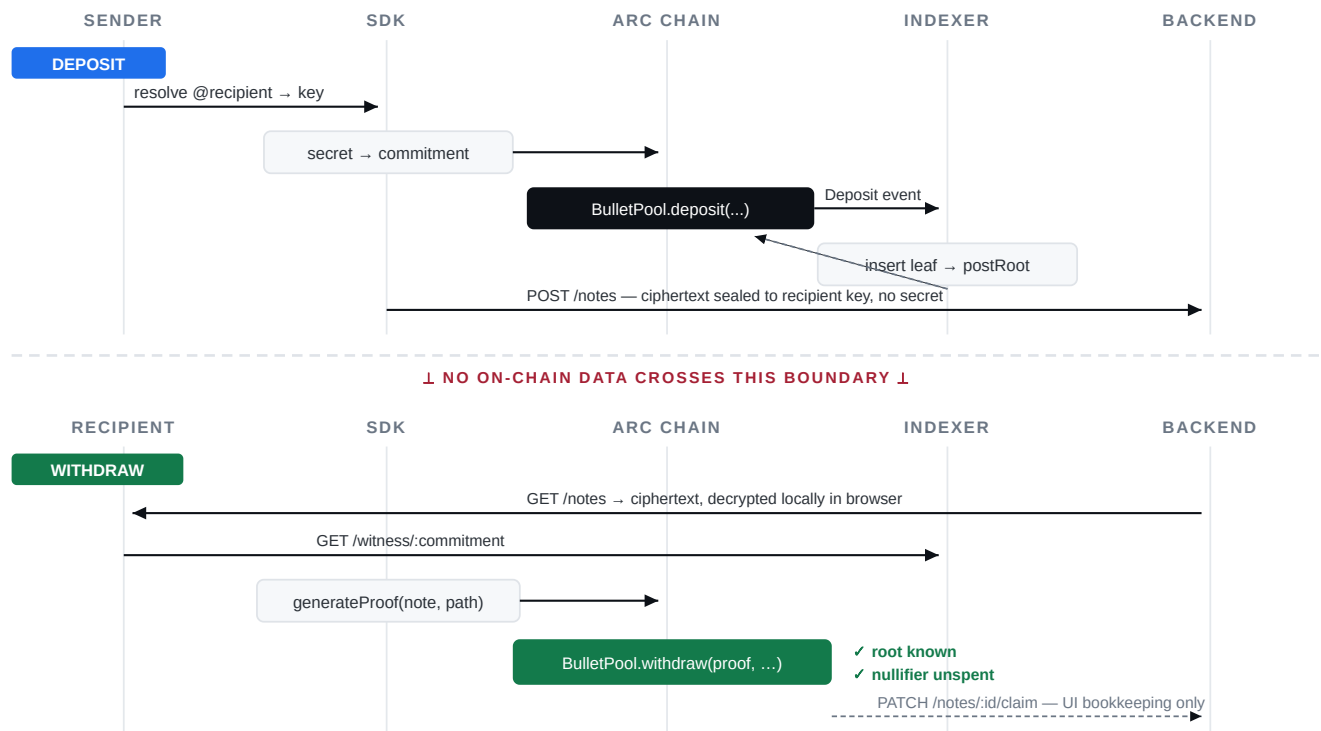


Figure 3 — End-to-end sequence. The recipient's browser is the only place a secret exists in plaintext.

Deposit

```
SDK      secret → commitment
          encrypt note to hushhPublicKey
Chain    BulletPool.deposit(
          token, amount, commitment)
Indexer  insert leaf → postRoot
Backend  POST /notes (ciphertext only)
```

Withdraw

```
Backend  GET /notes → decrypt locally
Indexer  GET /witness/:commitment
SDK      generateProof(note, merklePath)
Chain    BulletPool.withdraw(proof, root,
          nullifier, recipientDigest,
          recipient, token, amount)
Backend  PATCH /notes/:id/claim
```

Note the asymmetry: the deposit transaction is signed by the sender, the withdraw transaction by anyone holding the proof. Because `recipientDigest` is bound as a public input, a proof observed in the mempool cannot be front-run and redirected — the payout address is fixed at proving time.

07 Cryptographic construction

Only the withdraw path requires a circuit. A deposit is a commitment, an ERC-20 transfer, and an event — all publicly verifiable without zero knowledge.

Commitment scheme

```
nullifier = Poseidon([ secret ])
commitment = Poseidon([ secret, recipientDigest, amount, tokenHash ])
tokenHash = uint256(uint160(tokenAddress))
```

Poseidon is used rather than Keccak because it is arithmetization-friendly over BN254: a Keccak-based Merkle path would cost orders of magnitude more constraints, pushing browser proving time from seconds into minutes.

The withdraw circuit

Public inputs (locked): [root, nullifier, recipientDigest, amount, tokenHash]

Private witness: secret, Merkle path elements, path index bits.

The circuit enforces three statements simultaneously:

1. **Correct derivation.** The commitment is the Poseidon hash of the private secret together with the public recipientDigest, amount, and tokenHash.
2. **Membership.** Hashing that commitment upward along the private Merkle path with the private index bits reproduces the public root.
3. **Nullifier binding.** The public nullifier is the Poseidon hash of the same private secret used in (1) — so a note cannot be spent twice under different nullifiers.

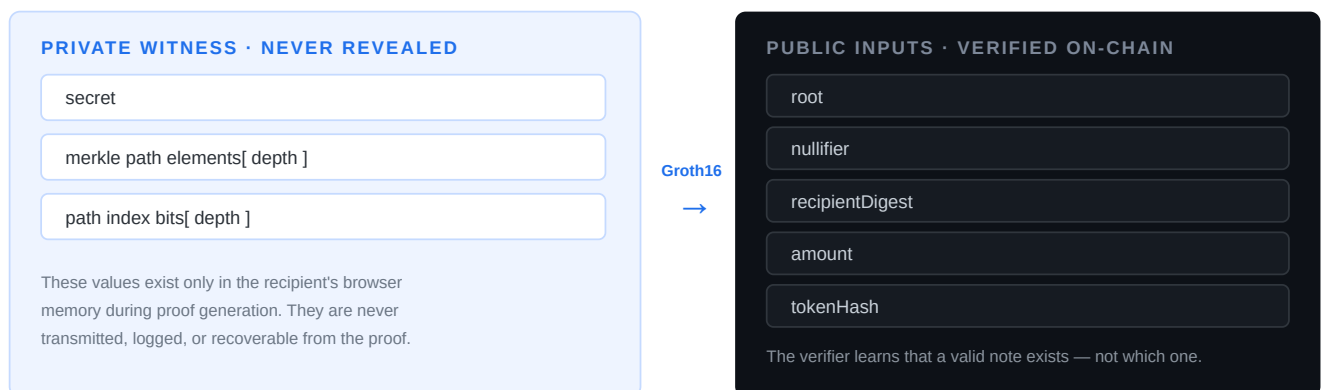


Figure 4 — Circuit interface. The proof reveals membership without revealing the leaf.

Proof system selection

CHOICE	RATIONALE	COST ACCEPTED
Groth16	Constant-size proofs and the cheapest known on-chain pairing verification — the dominant factor in per-withdrawal gas	Circuit-specific trusted setup required
BN254	Native EVM precompiles for pairing and curve arithmetic; no expensive in-contract emulation	~100-bit effective security, below BLS12-381
Poseidon	Low constraint count inside the circuit, enabling practical in-browser proving without a server	Less battle-tested than Keccak
Fixed denominations	Amount-matching resistance without the complexity of encrypted balances	Amounts are public; UX is quantized

08 Contracts & state machine

CONTRACT	ROLE
BulletPool.sol	Custody of deposited notes; deposit and withdraw entrypoints; the nullifier set; an embedded registry of accepted tokens
MerkleRootManager.sol	A rolling window of the 64 most recent valid roots, with <code>isKnownRoot()</code> consulted at withdraw time
WithdrawVerifier.sol	Groth16 verifier generated automatically at circuit build; replaces the mock verifier after <code>zk:build</code>
MockUSDC.sol	Test ERC-20 standing in for USDC during testnet operation

Why a 64-root window

A naive design accepts only the single current root. That design is broken in practice: between the moment a recipient fetches their witness and the moment their transaction is mined, any other user's deposit advances the root and invalidates the proof — producing a race the recipient loses whenever the pool is busy, which is exactly when they most want to transact.

The rolling window resolves this. A witness stays valid across subsequent deposits for as long as its root remains inside the window, decoupling proof generation from block timing. The window is bounded rather than unbounded so that a root cannot be replayed indefinitely after the indexer set has changed.

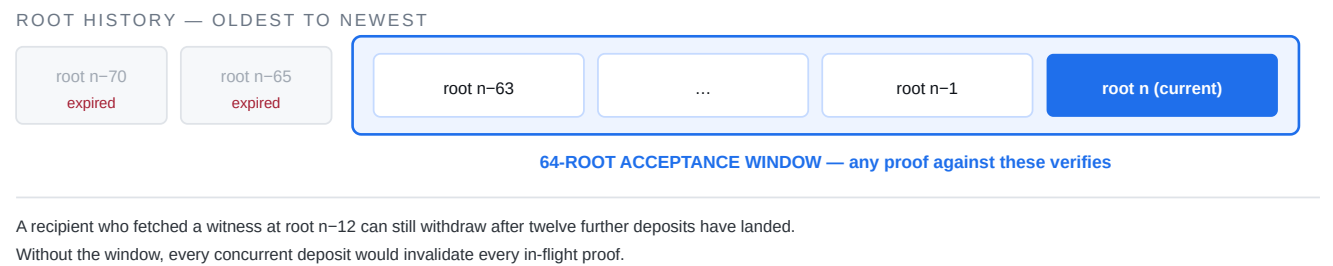


Figure 5 — The rolling root window decouples proving from block timing.

Withdraw validation sequence

The pool rejects the transaction unless every check passes, in order:

- `isKnownRoot(root)` — the claimed root is inside the 64-entry window.
- `!nullifiers>nullifier]` — this note has not already been spent.
- `verifyProof(...)` — the Groth16 pairing check succeeds against the locked public inputs.
- The token is present in the registry and the amount matches an accepted denomination.
- Effects: mark the nullifier spent, *then* transfer — nullifier before transfer, so a reentrant call finds the note already consumed.

09 Trust model & threat analysis

The security question worth asking of any privacy system is not "is it encrypted" but "what does each party gain by defecting". The table below answers that for every component Hushh depends on.

PARTY	STEAL FUNDS	CENSOR	DEANONYMIZE	NOTES
Indexer / relay	No	Yes	No	Can withhold roots or witnesses, stalling withdrawals. Cannot forge a leaf, since the pool accepts only roots consistent with its own deposit events.
Backend	No	Yes	Partial	Holds ciphertext and routing metadata. Learns who messages whom and when — not amounts, not secrets, and not the on-chain link.
Contract owner	No	Registry	No	Can add or remove accepted tokens. No withdrawal authority and no access to pooled funds.
Trusted setup	If compromised	No	No	Toxic-waste retention permits forged proofs — that is theft from the pool, not deanonymization of users.
Chain validators	No	Yes	No	Standard L1 censorship assumptions. Proof contents remain opaque to block producers.
Network observer	No	No	Timing	Can correlate deposit and withdraw timing in a thin pool. Mitigated by pool depth and by delaying withdrawal.

The invariant that matters: custody and unlinkability rest exclusively on the contracts and the circuit. Every off-chain component is a *liveness* dependency. A malicious indexer and a malicious backend, colluding, can stop Hushh from working — they cannot take a user's funds and cannot reconstruct the deposit-to-withdraw link.

Specific attacks and mitigations

ATTACK	MITIGATION
Double-withdraw	The nullifier is deterministically derived from the secret. Two withdrawals of one note necessarily produce the same nullifier, and the second is rejected by the spent-set check.
Proof front-running	<code>recipientDigest</code> is a locked public input. A proof lifted from the mempool pays only the address it was generated for; redirecting it invalidates the pairing check.
Forged Merkle root	Roots are constrained by observable on-chain <code>Deposit</code> events. Any party can independently rebuild the tree and detect a root inconsistent with the event log.
Reentrancy on withdraw	The nullifier is written before the ERC-20 transfer, so a reentrant callback encounters an already-spent note.
Amount fingerprinting	Fixed denominations only. A withdrawal of a non-standard amount — the classic correlation vector — is not expressible.
Backend key substitution	Public keys resolved from the registry should be verified against the counterparty out of band for high-value payments. A malicious registry could otherwise serve its own key and intercept ciphertext — though not funds, which remain gated by the on-chain proof.

10 Honest privacy limits

Stated without hedging, because a privacy protocol that overclaims is more dangerous than one that does not exist — users calibrate real-world risk against these sentences.

- **Unlinkability, not confidentiality.** Deposit and withdraw are unlinked. Amounts are standardized into fixed denominations, not encrypted. This is not a full shielded pool with hidden balances.
- **Anonymity scales with pool size.** A thin pool is a weak anonymity set. At demo-stage volume the set is small, and a determined observer with timing data may narrow it substantially.
- **Root posting is permissioned.** A single relayer posts roots today. It cannot steal or deanonymize, but it can stall withdrawals indefinitely.
- **Trusted setup is single-contributor.** A retained toxic-waste parameter would permit forged proofs — theft from the pool, not deanonymization. An MPC ceremony is a prerequisite for mainnet.
- **Off-chain metadata leaks.** Claim links, email delivery, and server logs can leak timing and correspondence patterns even while on-chain unlinkability holds fully.
- **Timing correlation is real.** A deposit followed minutes later by a same-denomination withdrawal in an otherwise quiet pool is a weak but genuine signal. Waiting is a meaningful defence.
- **The circuit is unaudited.** No third-party review has been completed. Treat all deployed code as experimental.

11 Deployment

Network

Network	Arc Testnet
Chain ID	5042002
Currency	USDC
RPC	rpc.testnet.arc.network
Explorer	testnet.arcscan.app

Services

App	hush-protocol-zeta.vercel.app
Backend	hush-protocol-backend.onrender.com
Indexer	hush-protocol-indexer.onrender.com

Requirements

Node 22+ · pnpm 9+ · Circom (circuit builds) · Postgres (indexer)

Contract addresses

CONTRACT	ADDRESS
BulletPool	0x9E65584C6ACE76cEFde4FD5f50E6bB9763999dCe
MerkleRootManager	0x6D6530A1fC908792DA2aa533a5F299283E6F062a
BulletVerifier	0x0A95c430Bf6AA3140A4Bf04C8D5982472331008d
MockUSDC	0x7d7aF5715e5671e0E3126b2428Dc2629bD9061e3

12 Roadmap

The sequence is deliberate: decentralize the components that can stall the system before adding features that widen its surface area.

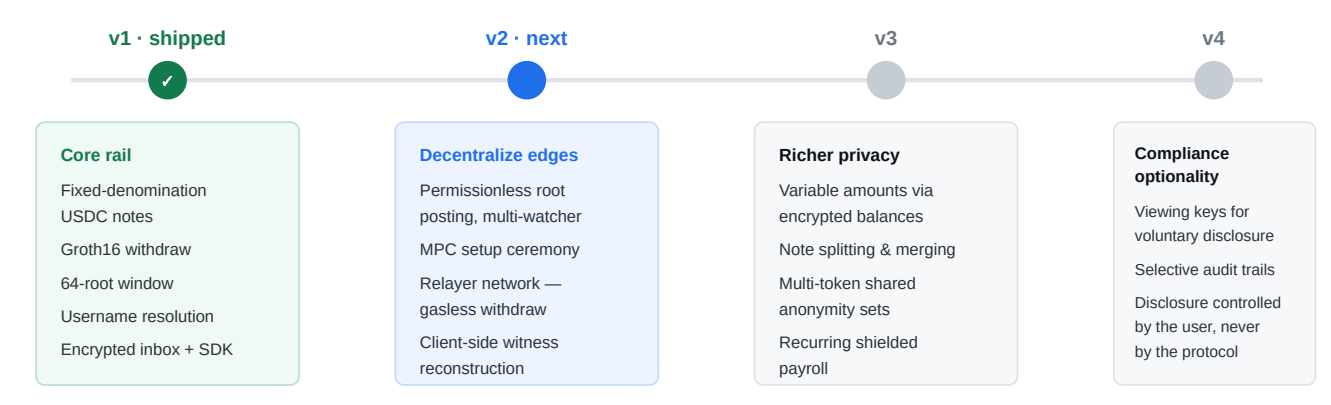


Figure 6 — Development sequence. Liveness decentralization precedes feature expansion.

13 Design principles

PRINCIPLE	WHY IT IS ENFORCED
Never reimplement crypto in the app	The frontend calls the SDK. Poseidon and proving exist in exactly one place, so a fix lands once and an audit has one target.
The server holds ciphertext, never secrets	A full backend compromise must leak metadata only — never funds, never identities, never the on-chain link.
Safety on-chain, liveness off-chain	Anything off-chain may stall the system. Nothing off-chain may break it. This is the line every architectural decision is checked against.
State the limits	Users cannot calibrate real-world risk against marketing language. Section 10 exists for that reason and is maintained as a first-class part of this document.

Hushh Protocol is testnet software with an unaudited circuit and a single-contributor trusted setup. Do not use it to protect information whose disclosure you cannot afford. Nothing in this document is financial, legal, or security advice.